# 15 — Python-Programme

Vom Skript zum strukturierten Programm

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

# Agenda

1. Warum Struktur? – vom Skript zum Programm
2. Funktionen als Bausteine
3. `main()` – der Dirigent
4. Der Startblock: `if __name__ == "__main__":`
5. Die 5 Bausteine einer `.py`-Datei
6. **Klassiker: Hangman** – live programmiert

**Lernziel**: Aus einzelnen Code-Schnipseln ein **richtiges Programm** machen – und verstehen, **wann** welcher Code wirklich ausgeführt wird.

**Heute**: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 15).

# Warum Struktur? Das Spaghetti-Problem

```
shop = {"Eco-Sneaker": 89.95, "Hemp-High": 109.00}
for p, preis in shop.items():
    print(f"{p}: {preis} EUR")
gesamt = 0
for preis in shop.values():
    gesamt = gesamt + preis
print(f"Lagerwert: {gesamt} EUR")
```

- Alles auf einer Ebene – liest sich wie eine Wäscheliste
- Nicht wiederverwendbar, nicht einzeln testbar, beim Import läuft **alles sofort los**

**Faustregel**: Wird ein Skript länger als ca. 20 Zeilen, braucht es Struktur.

# Funktionen als Bausteine

```python
def bestand_anzeigen(shop):
    for p, preis in shop.items():
        print(f"{p}: {preis} EUR")

def lagerwert(shop):
    return sum(shop.values())
```

- Jede Funktion hat **genau eine Aufgabe** (*Single Responsibility*)
- Der Name sagt, **was** passiert – einzeln testbar, wiederverwendbar

**“und”-Test**: Müssen Sie eine Funktion mit *“und”* beschreiben (`laden_und_anzeigen`), sind es vermutlich **zwei** Funktionen.

▶ **Live-Demo** – `01_spaghetti_vs_strukturiert.py`

# Funktionen rufen Funktionen auf

```python
def bericht(shop):
    """Lagerbericht fuer Veggie Soles."""
    print("=== Veggie Soles Bericht ===")
    bestand_anzeigen(shop)              # delegiert
    print(f"Wert: {lagerwert(shop):.2f} EUR")
```

- `bericht` **delegiert** – es kennt die Details der Anzeige nicht
- Das ist **Abstraktion**: jede Schicht liest sich auf ihrer eigenen Flughöhe

▶ **Live-Demo** – `02_funktionen_als_bausteine.py`

✎ **Sofort ausprobieren** – Notebook 15, Kap. 1–2: unstrukturierten Code in Funktionen zerlegen

## `main()` – der Dirigent

```
def main():
    """Hauptprogramm: Veggie-Soles-Bericht."""
    shop = {"Eco-Sneaker": 89.95,
            "Hemp-High": 109.00}
    bericht(shop)

main()   # im Notebook so -- Startblock kommt gleich
```

- **Steuert den Ablauf**: ruft Hilfsfunktionen der Reihe nach auf
- Enthält **selbst wenig Logik** – die steckt in den Bausteinen
- Liest sich wie die **Zusammenfassung** Ihres Programms

▶ **Live-Demo** – `03_main_funktion.py`

# Unter der Haube: warum `main()`?

```
print("A")          # 1) Modulebene: laeuft SOFORT
def gruss():
    print("B")      # 2) nur definiert (FS 07!)
print("C")          # 3) laeuft sofort
gruss()             # 4) JETZT erst laeuft B
```

| Zeile | Python tut | Ausgabe |
|---|---|---|
| `print("A")` | sofort ausführen | `A` |
| `def gruss():` | nur Funktionsobjekt anlegen | – |
| `print("C")` | sofort ausführen | `C` |
| `gruss()` | Sprung in den Block | `B` |

Modulebene läuft **sofort** – `main()` bündelt alles in **einem** Anrufplan.

# Was gehört in `main()` – und was nicht?

| Gehört in `main()` | Gehört NICHT in `main()` |
|---|---|
| Programmablauf steuern | Funktionsdefinitionen |
| Hilfsfunktionen aufrufen | Imports, Konstanten |
| Anfangsdaten, Eingaben | Detail-Rechenlogik |
| Ergebnisse ausgeben | loser Code auf Modulebene |

**Test**: Wer nur `main()` liest, versteht, **was** das Programm tut – ohne das **Wie** zu kennen.

✎ **Sofort ausprobieren** – Notebook 15, Kap. 3: eine `main()` für die Notenauswertung schreiben

# Der Startblock: `if __name__ == "__main__":`

```python
# shop.py
def main():
    print("Shop startet ...")

if __name__ == "__main__":    # Startblock
    main()
```

- `__name__` ist eine Variable, die **Python selbst setzt** – noch bevor Ihre erste Zeile läuft
- Der Startblock steht **immer ganz unten** und ist der einzige lose Code der Datei
- Er entscheidet: Programm starten – oder nur Funktionen bereitstellen?

# Unter der Haube: was ist __name__ wirklich?

Dieselbe Datei, zwei Aufrufarten:

```python
# begruessung.py
print(f"__name__ = {__name__}")
```

| Aufruf | Python setzt vorher | Ausgabe |
|---|---|---|
| `python3 begruessung.py` | `__name__ = "__main__"` | `__name__ = __main__` |
| `import begruessung` | `__name__ = "begruessung"` | `__name__ = begruessung` |

**Deshalb** schützt der Startblock den Startcode: beim direkten Start ist der Vergleich **wahr** → `main()` läuft. Beim Import ist er **falsch** → nur die Definitionen werden geladen.

# Skript vs. Modul – eine Datei, zwei Rollen

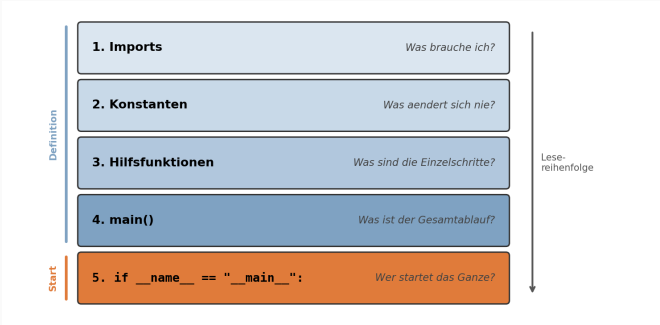
Mit Startblock kann **dieselbe Datei beides sein**:

| Aspekt | Skript (direkt) | Modul (importiert) |
|---|---|---|
| `__name__` | `"__main__"` | `"shop_utils"` |
| Zweck | Programm starten | Funktionen bereitstellen |
| Start | `python3 shop_utils.py` | `import shop_utils` |

▶ **Live-Demo** – `04_modul_oder_skript.py`

✎ **Sofort ausprobieren** – Notebook 15, Kap. 4: `__name__` untersuchen (im Notebook immer `"__main__"`!)

# Die 5 Bausteine einer .py-Datei




Die Reihenfolge ist **nicht zufällig**: Was die Funktionen brauchen (Imports, Konstanten), steht oben – was alles startet, ganz unten.

▶ **Live-Demo** – `05_veggie_soles_komplett.py` (alle 5 Bausteine in einer Datei)

# Notebook vs. `.py`-Datei

| Aspekt | Notebook | `.py`-Datei |
|---|---|---|
| Ausführung | Zellen, freie Reihenfolge | sequenziell von oben |
| Zustand | bleibt zwischen Zellen | frisch bei jedem Start |
| `__name__` | immer `"__main__"` | `"__main__"` oder Modulname |
| Ideal für | Lernen, Experimente | Programme, Tools, Module |

**Das Notebook ist die Werkbank, die `.py`-Datei das Produkt.** Im Notebook: `main()` direkt aufrufen – in der `.py`-Datei den Startblock ergänzen.

# Anti-Patterns – was Sie vermeiden

| Anti-Pattern | Warum schlecht? |
|---|---|
| Loser Code auf Modulebene | läuft beim Import sofort los |
| Globale Variablen statt Parameter | versteckte Abhängigkeiten |
| Eine riesige `tue_alles()`-Funktion | unlesbar, nichts wiederverwendbar |
| `print()` statt `return` in Berechnungen | Aufrufer kann nichts weiterverwenden |

✎ **Sofort ausprobieren** – Notebook 15, Kap. 7: die vier Strukturfehler finden und korrigieren

# Klassiker: Hangman – die Aufgabe

- **Spieler 1** tippt das Wort ein – `print("\n" * 50)` versteckt es
- **Spieler 2** rät Buchstaben, **6 Leben** – Fehlgriff kostet eins
- Anzeige pro Zug: Geratenes sichtbar, Rest _, dazu die Leben

**So soll es laufen**

```
Wort (Spieler 1): beere   <- danach 50 Leerzeilen
Wort: _____   Leben: 6
Buchstabe: e
Wort: _ee_e   Leben: 6
Buchstabe: x
Wort: _ee_e   Leben: 5
...
Gewonnen! Das Wort war: beere
```

# Unter der Haube: Zustand + Schleife = Spiel

Drei Variablen tragen das ganze Spiel – die Hauptschleife verändert sie, die Abbruchbedingung prüft sie:

| Zug | `geraten` | `leben` | Anzeige |
|---|---|---|---|
| Start | `set()` | 6 | `_____` |
| `"e"` (Treffer) | `{"e"}` | 6 | `_ee_e` |
| `"x"` (daneben) | `{"e", "x"}` | 5 | `_ee_e` |

- Gewonnen: Anzeige == Wort · Verloren: `leben == 0`
- Genau das Muster von **Zahlenraten (FS 10)** – verallgemeinert: **jedes** interaktive Programm ist *Zustand in Variablen + Hauptschleife + Abbruchbedingung*

# Klassiker: Hangman – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_hangman.py`
Teilziele:

1. Spielzustand definieren: `wort`, `geraten` (Set), `leben`
2. `zeige_wort(wort, geraten)` als eigene Funktion
3. Hauptschleife mit Gewonnen-/Verloren-Abbruch
4. Buchstaben-Logik: im Wort? sonst Leben runter
5. Alles ordnen: `main()` + Startblock

✎ **Zum Selberlösen** – Handout *“Klassiker 15: Hangman”* (`Klassiker/15_Klassiker_Hangman.pdf`). Vertiefung: Notebook 15, Abschlussübungen.

# Cheat Card

| Aufgabe | Syntax |
|---|---|
| Funktion definieren | `def name(parameter):` |
| Hauptfunktion | `def main():` |
| Startblock | `if __name__ == "__main__": main()` |
| Konstante | `MAX_LEBEN = 6` |
| Docstring | `"""Beschreibung."""` |
| 5 Bausteine | Imports > Konstanten > Hilfsfkt. > `main()` > Startblock |
| Klassiker heute | Hangman: Zustand + Hauptschleife + `main()` |

**Faustregel**: Kein loser Code auf Modulebene – alles in Funktionen, gestartet über den Startblock. Und: Wer nur `main()` liest, muss das Programm verstehen.

# Ausblick: Notebook 16 – Pandas

Unser Hangman verwaltet drei Werte – echte Programme verwalten **Tausende**:

- Bestellungen, Kunden, Preise: Daten kommen **tabellarisch**
- Dafür gibt es ein Profi-Werkzeug: **`pandas`** und der **DataFrame**
- Erste externe Bibliothek: `import pandas as pd` – das `import`-Rezept kennen Sie schon

Auch Datenanalyse-Programme profitieren von `main()` und den 5 Bausteinen – die heutige Struktur bleibt Ihr Werkzeug.

# Heute geübt

- ✓ Spaghetti-Code von strukturiertem Programm unterschieden
- ✓ Funktionen als Bausteine: *Single Responsibility*, Abstraktion
- ✓ `main()` als Dirigent – und **warum**: Modulebene läuft sofort
- ✓ `__name__`: von Python gesetzt – `"__main__"` oder Modulname
- ✓ Startblock `if __name__ == "__main__":` – Skript **und** Modul
- ✓ Die **5 Bausteine** einer `.py`-Datei, Anti-Patterns
- ✓ **Klassiker Hangman**: Zustand + Hauptschleife + `main()`

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Hangman** eigenständig lösen (Handout)
- Notebook 15 durcharbeiten, Abschlussübungen (z. B. Lohnabrechnung refaktorisieren)